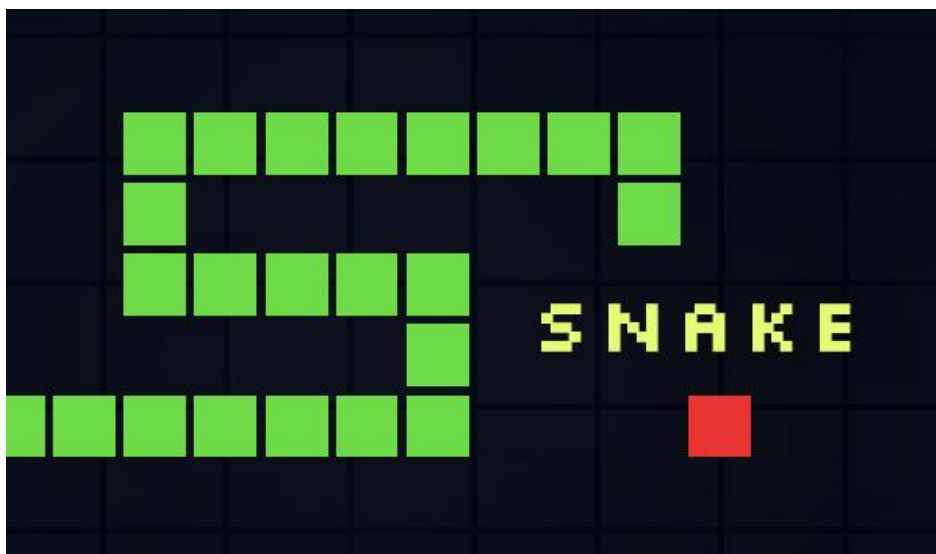


Projeto: OOPS – Jogo com Figuras Geométricas



Programação Orientada a Objetos

Professor: José Valente de Oliveira

Professor: José Barateiro

Trabalho realizado por Grupo N°1:

Catarina Machado a58473

Diogo Luz a65873

João Morales a75445

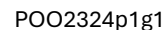
2º Semestre – 2023/2024

Índice

Conteúdo

Diagrama de classes UML de Implementação	3
Descrição das classes e as suas responsabilidades	4
Modo de Utilização.....	7
Opções de Projeto Tomadas	9
Modo de Utilização.....	12
Opções de Projeto Tomadas (Padrões de Projeto)	14
Padrões de Projeto	15
Limitações.....	16
Testes Unitários JUnit	16

3



Descrição das classes e as suas responsabilidades

Pacote SnakeGame

1. Food.java

Gerência a comida que a cobra come, incluindo sua criação, posicionamento aleatório e desenho na tela. A classe controla se a comida é um círculo ou um quadrado e garante que a comida não apareça em locais inválidos, como sobre a cobra ou obstáculos.

2. GameArenaTxt.java

Gerência a arena do jogo em modo texto. Implementa o loop principal do jogo, lida com entradas do usuário para mover a cobra e verifica colisões. Também desenha a arena e atualiza o estado do jogo.

3. GameOver.java

Gerência a lógica de fim de jogo, incluindo a captura do nome do jogador para registrar a pontuação e a exibição das pontuações mais altas.

4. Grid.java

Representa a grade do jogo. A classe é responsável por inicializar a grade, definir e obter estados dos tiles (como se um tile contém comida, parte da cobra ou um obstáculo), e desenhar a grade.

5. Highscore.java

Gerência as pontuações altas. Lê e escreve pontuações em um arquivo, classifica as pontuações e exibe as pontuações mais altas.

6. Main.java

Ponto de entrada do jogo. Inicializa a configuração do jogo e inicia a arena do jogo.

7. Obstaculo.java

Gerência os obstáculos no jogo. Permite a adição, remoção e posicionamento de obstáculos na grade, bem como a verificação de colisões.

8. Score.java

Mantém a pontuação atual do jogo. Atualiza a pontuação quando a cobra come comida, e desenha a pontuação na tela.

9. Snake.java

Representa a cobra e sua lógica de movimento. Controla o movimento da cobra, verifica colisões com paredes, obstáculos e o próprio corpo, e gerência a adição de novos segmentos ao corpo quando a cobra come comida.

10. SnakeBody.java

Gerência os segmentos do corpo da cobra. Atualiza a posição dos segmentos conforme a cobra se move, e desenha os segmentos na tela.

11. SnakeHead.java

Gerência a cabeça da cobra. Controla o movimento da cabeça, verifica colisões com paredes e obstáculos, e desenha a cabeça na tela.

12. Tile.java

Representa um tile na grade do jogo. Mantém o estado de cada tile (se contém comida, parte da cobra ou um obstáculo) e fornece métodos para alternar estados e verificar colisões.

Pacote FigurasGeo

1. Circulo.java

Representa um círculo no jogo. Gerência propriedades como centro e raio, e fornece métodos para verificar colisões com outras figuras geométricas.

2. IFigurasGeometricas.java

Interface que define operações para figuras geométricas. Fornece métodos para calcular perímetro, verificar colisões, e gerenciar propriedades geométricas como ângulos e coordenadas.

3. Poligono.java

Representa um polígono com uma lista de vértices e arestas. Fornece métodos para calcular perímetro, verificar colisões com outros polígonos e manipular vértices.

4. Ponto.java

Representa um ponto genérico no espaço. Gerência coordenadas x e y, e fornece métodos para operações geométricas básicas como rotação e movimento.

5. Quadrado.java

Representa um quadrado no jogo. Herda de Polígono e fornece métodos específicos para manipulação de quadrados.

6. Reta.java

Representa uma reta definida por dois pontos. Fornece métodos para verificar colinearidade e calcular inclinação.

7. Retangulo.java

Representa um retângulo no jogo. Herda de Polígono e fornece métodos específicos para manipulação de retângulos.

8. SegmentoReta.java

Representa um segmento de reta, derivado da classe Reta. Fornece métodos para verificar interseção com outros segmentos de reta.

9. Triangulo.java

Representa um triângulo no jogo. Herda de Polígono e fornece métodos específicos para manipulação de triângulos.

Pacote GUI

- GameArenaView.java

Gerência a visualização da arena do jogo. Renderiza os componentes visuais do jogo, como a cobra, a comida e os obstáculos.

- GameConfigFrame.java

Configura a tela de configuração do jogo. Permite ao usuário definir parâmetros como tamanho da grade, tamanho da cobra, tipo de comida, e modo de entrada (automático ou manual).

- GameController.java

Controla a lógica do jogo e a interação do usuário. Gerência entradas do teclado para mover a cobra, atualiza o estado do jogo e verifica colisões.

- GameOverFrame.java

Exibe a tela de fim de jogo. Captura o nome do jogador para registrar a pontuação e exibe as pontuações mais altas.

- ScorePanel.java

Exibe o painel de pontuação. Mostra a pontuação atual do jogador e atualiza a exibição conforme a pontuação muda.

Modo de Utilização

Existem duas interações diferentes, dependendo do modo de jogo que desejamos jogar: um modo texto jogado na linha de comandos e outro modo gráfico com uma interface de usuário (GUI). Antes de iniciar o jogo, é perguntado qual modo de jogo você deseja inicializar: o modo gráfico (tecla 1) ou o modo de texto (tecla 2).

Modo de Texto

Neste modo, serão feitas perguntas para configurar o jogo. Caso você insira um valor não permitido, a pergunta será repetida. Sugere-se que o tamanho do jogo não seja inferior a 200x200 (10x10 tiles), pois tamanhos menores podem ser muito pequenos para serem jogáveis.

Os controles do jogo são as seguintes teclas:

"W" para subir,

"S" para descer,

"A" para seguir pela esquerda,

"D" para seguir pela direita.

No modo de entrada (input), é necessário pressionar "Enter" para que o comando seja reconhecido.

O jogo termina quando a cobra colide com uma parede, obstáculo ou com ela própria. No fim do jogo, você terá a possibilidade de salvar seu nome e sua pontuação na lista de melhores pontuações (highscores) do jogo.

Modo GUI

Neste modo de jogo, uma janela aparecerá com várias configurações para selecionar. É necessário selecionar todas para o jogo funcionar corretamente (o obstáculo é a exceção; se nada for escrito, nenhum obstáculo será desenhado).

Após selecionar as configurações, clique no botão "Iniciar Jogo" e o jogo começará. Assim como no modo texto, o jogo terminará quando a cobra colidir com um obstáculo, uma parede ou com ela própria.

Os controles do jogo são as seguintes teclas:

"Seta Cima" para subir,

"Seta Baixo" para descer,

"Seta Esquerda" para seguir pela esquerda,

"Seta Direita" para seguir pela direita.

No fim do jogo, uma nova janela aparecerá mostrando os 5 maiores resultados, a pontuação obtida e uma caixa de texto para inserir o nome a ser registado no highscore. Após o término do jogo, você terá a possibilidade de repetir o jogo ou fechá-lo.

Notas Gerais

- Modo de Texto: Recomendado um tamanho mínimo de 200x200 (10x10 tiles) para jogabilidade adequada.
- Modo Gui: Recomendado um tamanho mínimo de 200*200(10*10 tiles).
- Fim do Jogo: O jogo termina ao colidir com paredes, obstáculos ou com a própria cobra.
- Registo de Pontuação: Possibilidade de registar seu nome e pontuação nos highscores.
- Modo GUI: Requer seleção completa das configurações, exceto obstáculos, e apresenta uma interface amigável para visualização e controle.

Opções de Projeto Tomadas

Foram tomadas várias decisões relativas ao desenvolvimento do projeto. Tais serão representadas em baixo:

- Utilizou-se um padrão Model View Controller(MVC) para fazer a interação gráfica do utilizador, com as classes GameConfigFrame, GameController e GameArenaView.
- A utilização do modelo strategy para escolher o tipo de comida (quadrado ou um círculo).
- Usar uma variável booleana – isFoodCircle – para decidir se os objetos de comida inicializados são representados por quadrados (true) ou por círculos (false);
- Usar uma variável booleana – isFullRender – para decidir se as funções de draw vão desenhar apenas o contorno dos objetos (false) ou os objetos completos (true);
- Usar uma variável booleana – automaticInput – para decidir o movimento no próximo loop, quer seja simulado (true) ou quer seja dado pelo utilizador;
- Usar uma variável booleana – isDynamic – para decidir se os obstáculos criados têm movimento (true) ou são estáticos.
- O utilizador só vai poder escolher 3 tamanhos de cabeça de cobra, 1, 4 ou 9. Estes valores representam o número de Tiles que vão ser representadas por uma unidade de cobra.
- A grid é a classe responsável por guardar a informação de jogo e imprimir/desenhar essa informação. Todas as outras classes trabalham de modo a manipular a informação da grid.
- Existem dois tipos de coordenadas a serem usadas neste jogo, coordenadas Pixel e coordenadas Índice. A grid maioritariamente usa coordenadas índice (facilita as contas e os loops), todas as outras classes usam coordenadas Pixel.
- O tileSize é uma variável usada pelo programa todo e que não varia e guarda o tamanho de uma tile, neste caso é 20 pixéis.
- A snake é representada por 3 classes, em que a classe Snake quase que funciona com uma manager das classes SnakeHead e SnakeBody.
- O movimento da snake é feito da seguinte maneira, a cabeça mexe-se na direção pretendida e todo o corpo segue a cabeça, sendo que a primeira parte do corpo segue a cabeça, e as restantes, uma a uma, seguem a próxima parte.
- No fim do jogo, ou “Game Over”, foi decidido que só iríamos apresentar os primeiros 5 resultados, juntamente com o resultado do jogo jogado.

Representa um retângulo no jogo. Herda de Polígono e fornece métodos específicos para manipulação de retângulos.

10. SegmentoReta.java

Representa um segmento de reta, derivado da classe Reta. Fornece métodos para verificar interseção com outros segmentos de reta.

11. Triangulo.java

Representa um triângulo no jogo. Herda de Polígono e fornece métodos específicos para manipulação de triângulos.

Pacote GUI

- GameArenaView.java

Gerência a visualização da arena do jogo. Renderiza os componentes visuais do jogo, como a cobra, a comida e os obstáculos.

- GameConfigFrame.java

Configura a tela de configuração do jogo. Permite ao usuário definir parâmetros como tamanho da grade, tamanho da cobra, tipo de comida, e modo de entrada (automático ou manual).

- GameController.java

Controla a lógica do jogo e a interação do usuário. Gerência entradas do teclado para mover a cobra, atualiza o estado do jogo e verifica colisões.

- GameOverFrame.java

Exibe a tela de fim de jogo. Captura o nome do jogador para registrar a pontuação e exibe as pontuações mais altas.

- ScorePanel.java

Exibe o painel de pontuação. Mostra a pontuação atual do jogador e atualiza a exibição conforme a pontuação muda.

Modo de Utilização

Existem duas interações diferentes, dependendo do modo de jogo que desejamos jogar: um modo texto jogado na linha de comandos e outro modo gráfico com uma interface de usuário (GUI). Antes de iniciar o jogo, é perguntado qual modo de jogo você deseja inicializar: o modo gráfico (tecla 1) ou o modo de texto (tecla 2).

Modo de Texto

Neste modo, serão feitas perguntas para configurar o jogo. Caso você insira um valor não permitido, a pergunta será repetida. Sugere-se que o tamanho do jogo não seja inferior a 200x200 (10x10 tiles), pois tamanhos menores podem ser muito pequenos para serem jogáveis.

Os controles do jogo são as seguintes teclas:

"W" para subir,

"S" para descer,

"A" para seguir pela esquerda,

"D" para seguir pela direita.

No modo de entrada (input), é necessário pressionar "Enter" para que o comando seja reconhecido.

O jogo termina quando a cobra colide com uma parede, obstáculo ou com ela própria. No fim do jogo, você terá a possibilidade de salvar seu nome e sua pontuação na lista de melhores pontuações (highscores) do jogo.

Modo GUI

Neste modo de jogo, uma janela aparecerá com várias configurações para selecionar. É necessário selecionar todas para o jogo funcionar corretamente (o obstáculo é a exceção; se nada for escrito, nenhum obstáculo será desenhado).

Após selecionar as configurações, clique no botão "Iniciar Jogo" e o jogo começará. Assim como no modo texto, o jogo terminará quando a cobra colidir com um obstáculo, uma parede ou com ela própria.

Os controles do jogo são as seguintes teclas:

"Seta Cima" para subir,

"Seta Baixo" para descer,

"Seta Esquerda" para seguir pela esquerda,

"Seta Direita" para seguir pela direita.

No fim do jogo, uma nova janela aparecerá mostrando os 5 maiores resultados, a pontuação obtida e uma caixa de texto para inserir o nome a ser registado no highscore. Após o término do jogo, você terá a possibilidade de repetir o jogo ou fechá-lo.

Notas Gerais

- Modo de Texto: Recomendado um tamanho mínimo de 200x200 (10x10 tiles) para jogabilidade adequada.
- Modo Gui: Recomendado um tamanho mínimo de 200*200(10*10 tiles).
- Fim do Jogo: O jogo termina ao colidir com paredes, obstáculos ou com a própria cobra.
- Registo de Pontuação: Possibilidade de registar seu nome e pontuação nos highscores.
- Modo GUI: Requer seleção completa das configurações, exceto obstáculos, e apresenta uma interface amigável para visualização e controle.

Opções de Projeto Tomadas (Padrões de Projeto)

Foram tomadas várias decisões relativas ao desenvolvimento do projeto. Tais serão representadas em baixo:

- Utilizou-se um padrão Model View Controller(MVC) para fazer a interação gráfica do utilizador, com as classes GameConfigFrame, GameController e GameArenaView.
- A utilização do modelo strategy para escolher o tipo de comida (quadrado ou um círculo).
- Usar uma variável booleana – isFoodCircle – para decidir se os objetos de comida inicializados são representados por quadrados (true) ou por círculos (false);
- Usar uma variável booleana – isFullRender – para decidir se as funções de draw vão desenhar apenas o contorno dos objetos (false) ou os objetos completos (true);
- Usar uma variável booleana – automaticInput – para decidir o movimento no próximo loop, quer seja simulado (true) ou quer seja dado pelo utilizador;
- Usar uma variável booleana – isDynamic – para decidir se os obstáculos criados têm movimento (true) ou são estáticos.
- O utilizador só vai poder escolher 3 tamanhos de cabeça de cobra, 1, 4 ou 9. Estes valores representam o número de Tiles que vão ser representadas por uma unidade de cobra.
- A grid é a classe responsável por guardar a informação de jogo e imprimir/desenhar essa informação. Todas as outras classes trabalham de modo a manipular a informação da grid.
- Existem dois tipos de coordenadas a serem usadas neste jogo, coordenadas Pixel e coordenadas Índice. A grid maioritariamente usa coordenadas índice (facilita as contas e os loops), todas as outras classes usam coordenadas Pixel.
- O tileSize é uma variável usada pelo programa todo e que não varia e guarda o tamanho de uma tile, neste caso é 20 pixéis.
- A snake é representada por 3 classes, em que a classe Snake quase que funciona com uma manager das classes SnakeHead e SnakeBody.
- O movimento da snake é feito da seguinte maneira, a cabeça mexe-se na direção pretendida e todo o corpo segue a cabeça, sendo que a primeira parte do corpo segue a cabeça, e as restantes, uma a uma, seguem a próxima parte.
- No fim do jogo, ou “Game Over”, foi decidido que só iríamos apresentar os primeiros 5 resultados, juntamente com o resultado do jogo jogado.

Padrões de Projeto

Padrão MVC (Model-View-Controller)

Descrição do Padrão:

O padrão MVC é um padrão de arquitetura de software que separa a aplicação em três componentes principais:

- **Model:** Gerencia a lógica dos dados da aplicação. Representa o estado do aplicativo que pode ser atualizado pela lógica de negócios e não está preocupado com a interface do usuário.
- **View:** Exibe os dados do Model ao usuário e envia as interações do usuário para o Controller.
- **Controller:** Recebe a entrada do usuário via View, processa essa entrada, manipulando os dados do Model e atualizando a View conforme necessário.

Aplicação no Projeto:

- **Model:** As classes Food, Snake, Score, Obstaculo e Grid representam a lógica de dados e estado do jogo.
- **View:** As classes GameArenaView, ScorePanel, e GameOverFrame cuidam da exibição dos dados do jogo.
- **Controller:** A classe GameController lida com a entrada do usuário e manipula o estado do jogo de acordo.

Padrão Strategy

Descrição do Padrão:

O padrão Strategy define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. O Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam.

Aplicação no Projeto:

A escolha entre a comida ser representada como um círculo ou um quadrado é feita usando o padrão Strategy. A decisão é feita no momento da criação da comida, e diferentes estratégias de desenho são aplicadas com base na forma escolhida.

Limitações

Interface de Usuário Simples: A maioria das funcionalidades parece ser implementada em modo texto, limitando a atratividade visual do jogo.

Gerenciamento de Colisões: A lógica de colisão pode não ser otimizada para desempenho em grades grandes ou com muitos obstáculos.

Movimento da Cobra: A movimentação automática da cobra pode se tornar previsível e limitada em inteligência.

Escalabilidade: A estrutura atual pode ser difícil de escalar para incluir novos recursos, como diferentes modos de jogo ou níveis de dificuldade variados.

Persistência de Dados: A persistência de dados para pontuações altas depende de arquivos de texto, o que pode ser menos robusto do que usar uma base de dados

Testes Unitários JUnit

Os JUnitTests serão anexados numa pasta dentro do ficheiro zipado.